

KONSPEKT PROJEKTU

Sieciowa gra „Rosyjska Ruletka” (klient-serwer)

Programowanie Sieciowe · CB 2026 · prowadzący: Janusz Gozdecki

Wykonawcy: Przemysław Rutyna, Piotr Skoczylas, Bartosz Toś

Cel projektu

Implementacja wieloosobowej, **współbieżnej** gry sieciowej w architekturze klient-serwer, w której serwer przeprowadza kolejne rundy rosyjskiej ruletki i eliminuje graczy aż do wyłonienia zwycięzcy. Projekt realizowany w języku **Python** z wykorzystaniem biblioteki **socket**.

Opis działania (zasady gry)

- Po starcie klient wyszukuje serwer w sieci lokalnej (multicast) lub podaje jego nazwę/adres, a następnie wprowadza pseudonim.
- Pseudonim nie może być pusty i może mieć maksymalnie 12 znaków (walidacja po stronie serwera).
- Gracze łączą się z serwerem (unicast TCP) przed rozpoczęciem rozgrywki.
- Liczba rund = liczba graczy – 1. Wyjątek: przy jednym kliencie rozgrywana jest jedna runda.
- W każdej rundzie serwer oddaje strzały do aktywnych graczy; prawdopodobieństwo zabójczego strzału wynosi 1/6.
- Runda kończy się wyeliminowaniem jednego gracza; wyeliminowany pozostaje połączony jako obserwator.
- Tryb wieloosobowy: wygrywa ostatni żywy gracz. Tryb jednoosobowy: wygrana, gdy pięć kolejnych strzałów jest pustych.

Realizacja wymagań projektu

- **Serwer współbieżny** — serwer obsługuje wielu klientów równocześnie; każde połączenie prowadzone jest w osobnym wątku (moduł *threading*) operującym na wspólnym stanie gry.
- **Adresacja multicast i unicast** — serwer cyklicznie rozgłasza dostępność usługi w grupie multicast (np. 239.0.0.1), dzięki czemu klient wykrywa go bez znajomości adresu IP; właściwa rozgrywka odbywa się przez unicast TCP.
- **Binarny format danych (TLV)** — wszystkie komunikaty kodowane są binarnie w formacie TLV (Type-Length-Value: 1 B typ, 2 B długość, N B wartość) pakowanym funkcją *struct.pack* (odpowiednik struktur języka C), w sieciowej kolejności bajtów (big-endian).
- **Tryb demona z logowaniem** — serwer uruchamiany jako proces w tle (detach/double fork); zdarzenia (połączenia, eliminacje, wyniki, błędy) zapisywane do logów systemowych przez moduł *syslog/logging*.
- **API do DNS** — klient może podać nazwę hosta serwera, rozwiązywaną na adres IPv4 funkcją *socket.getaddrinfo()*.
- **Koordinacja dostępu do wspólnych zasobów** — wspólny stan gry (lista aktywnych graczy, numer rundy, wyniki) przechowywany po stronie serwera i modyfikowany przez wielu klientów; dostęp synchronizowany muteksem (*threading.Lock*) chroniącym przed wyścigami.

Planowane moduły programowe

- **Moduł serwera (współbieżny)** — nasłuch unicast TCP oraz rozgłaszanie multicast; zarządzanie wątkami i listą aktywnych graczy; walidacja pseudonimów; synchronizacja wspólnego stanu (Lock); sterowanie rundami; logowanie do syslog; tryb demona.
- **Moduł klienta** — wyszukanie serwera (multicast) lub rozwiązanie nazwy (DNS); nawiązanie połączenia unicast; wprowadzanie pseudonimu; odbiór i prezentacja komunikatów oraz stanu gry, eliminacji i zwycięzcy.
- **Moduł logiki gry** — obsługa rund, losowanie strzałów z prawdopodobieństwem 1/6, eliminacja graczy, sprawdzanie warunków zakończenia.
- **Moduł protokołu (TLV)** — kodowanie i dekodowanie binarnych komunikatów (serializacja/deserializacja struct), wspólny dla klienta i serwera.

Protokoły, komunikaty i adresacja

- **Transport:** TCP/IP (unicast) dla rozgrywki; UDP multicast dla wyszukiwania usługi — komunikacja przez gniazda biblioteki socket.
- **Format ramki:** binarny TLV (struct.pack/unpack, big-endian).
- **Adresacja:** multicast — grupa np. 239.0.0.1 (ogłaszanie/wykrywanie serwera); unicast — serwer na adresie IPv4 (np. 127.0.0.1 w testach), klienci łączą się po adresie lub nazwie i numerze portu.
- **Typy komunikatów TLV:** ANNOUNCE/DISCOVER (multicast), JOIN, JOIN_ACK, START, SHOT, ELIMINATED, ROUND_END, WINNER, GAME_OVER, ERROR.

Planowany podział pracy

- **Przemysław Rutyna** — moduł serwera: współbieżność, tryb demona, logowanie, koordynacja dostępu do wspólnych zasobów.
- **Piotr Skoczylas** — moduł klienta: wyszukiwanie usługi przez multicast, obsługa DNS, interfejs użytkownika.
- **Bartosz Toś** — moduł logiki gry oraz binarny protokół TLV (kodowanie/dekodowanie).

Rezultat projektu

Działająca, współbieżna aplikacja klient-serwer (Python / socket) realizująca rozgrywkę rosyjskiej ruletki dla jednego lub wielu graczy, z wyszukiwaniem usługi przez multicast, binarnym protokołem TLV, serwerem w trybie demona z logowaniem do plików systemowych oraz synchronizowanym dostępem do wspólnego stanu gry i automatycznym wyłanianiem zwycięzcy.